2025/2026

FELCOM: A Secure Handheld Communication Device

Mini-Project

Prepared By:
Marc Gedeon
Yorgo Hassabou
Charbel Assaad

Presented To:
Dr. Hadi Jerdek

Table of Contents

Table of Figures

Abstract

FELCOM is a handheld radio that provides secure, text and voice communication in the 2.4 GHz ISM band. Each unit is built around an ESP32 microcontroller driving an nRF24L01+ transceiver, an INMP441 I²S microphone, a DAC, amplifier and speaker audio chain, an OLED display and tactile controls, all integrated on a custom PCB. To resist narrowband jamming and lower the probability of interception, the link continuously hops across six interference-free channels using Frequency-Hopping Spread Spectrum (FHSS). The nodes are kept aligned by a shared, timer-driven hopping schedule. A custom 32-byte packet format multiplexes text, voice and control traffic over the same radio and carries a compact forward-error-correction header. Reliability is provided by a layered FEC subsystem owned by the transceiver: a CRC-16 detects corruption on every protected packet, an automatic-repeat-request (ARQ) scheme guarantees exact delivery of chat messages, and an XOR block code reconstructs lost real-time audio packets without retransmission. Voice is captured as 8 kHz 8-bit PCM. Chat text is obfuscated with a lightweight XOR cipher. A built-in bit-error-rate (BER) test mode and live FEC counters allow link quality to be measured and analysed. The result demonstrates that robust, jam-resistant, low-cost digital voice and messaging can be realized entirely on commodity hardware.

1 Introduction

1.1 Background and Motivation

Reliable communication is usually taken for granted because it rests on a large, fixed infrastructure: cell towers, base stations, the Internet backbone. That infrastructure is also a single point of failure. In a natural disaster, in a remote area, or in any situation where the network is congested, censored or deliberately disabled, the same devices that work everywhere suddenly work nowhere. This motivates off-grid communication: a direct radio link between two people that depends on no third party.

A direct radio link, however, is exposed. A fixed-frequency transmitter is easy to locate, easy to intercept and trivial to jam, since a single interfering source on the right frequency is enough to silence it. The classical answer to this problem is spread spectrum. By rapidly and unpredictably changing the carrier frequency, a frequency-hopping system spreads its energy across a wide band: a narrowband jammer can only spoil the few hops that happen to land on its frequency, and an eavesdropper who does not know the hopping sequence sees only brief, scattered bursts. The same principle underlies Bluetooth today.

1.2 The Project

FELCOM applies these ideas on inexpensive, off-the-shelf hardware. It lets two units exchange text messages and live voice directly over the 2.4 GHz band. The link is made resilient by Frequency-Hopping Spread Spectrum, protected by a custom packet protocol with forward error correction, and the text is obfuscated by a lightweight cipher. Two extra modes, a real-time multiplayer Pong game and an RF/BER test screen, were added to validate, respectively, low-latency bidirectional traffic and raw link quality.

1.3 Objectives

The project set out to:

- build a working two-node handheld system on an ESP32 + nRF24L01+ platform.
- implement FHSS, with a synchronization mechanism robust to clock drift and to new nodes joining.
- define a flexible packet protocol able to multiplex text, voice and control traffic over one radio.
- add a layered forward-error-correction subsystem (detection, retransmission, and forward recovery) matched to each traffic type.
- provide a debug mode that measures packet loss and bit-error rate.
- integrate everything onto a custom PCB.

1.4 Technology and Methodology

The system is built on the Espressif ESP32 (dual-core, hardware timers, I^2S and DAC peripherals) paired with the Nordic nRF24L01+, a 2.4 GHz GFSK transceiver that

performs modulation in hardware and exposes 125 selectable 1 MHz channels making FHSS possible.

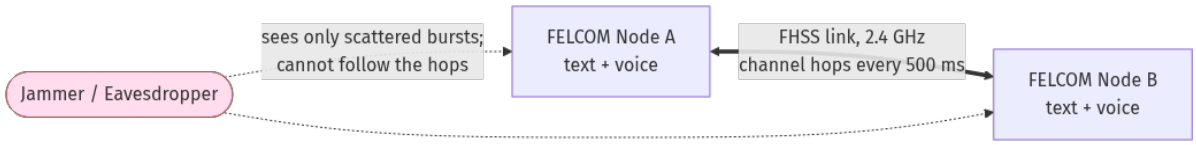

Figure 1: Two handhelds communicating directly over a frequency-hopping link.

2 System Overview

2.1 Hardware

Each FELCOM unit is identical and runs the same firmware (only a one-byte `NODE_ID` differs).

The core is an ESP32 DevKit V1. Around it:

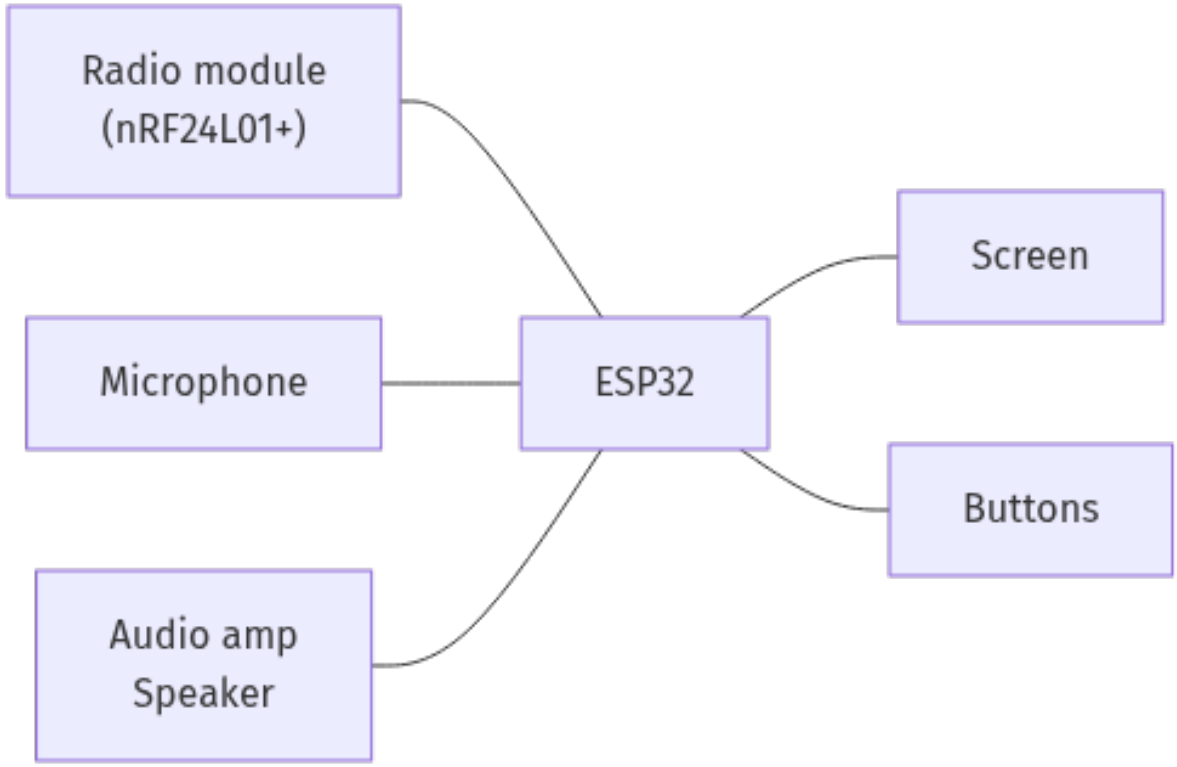

Figure 2: **ADD CAPTION**

The complete schematic and the routed PCB are shown below.

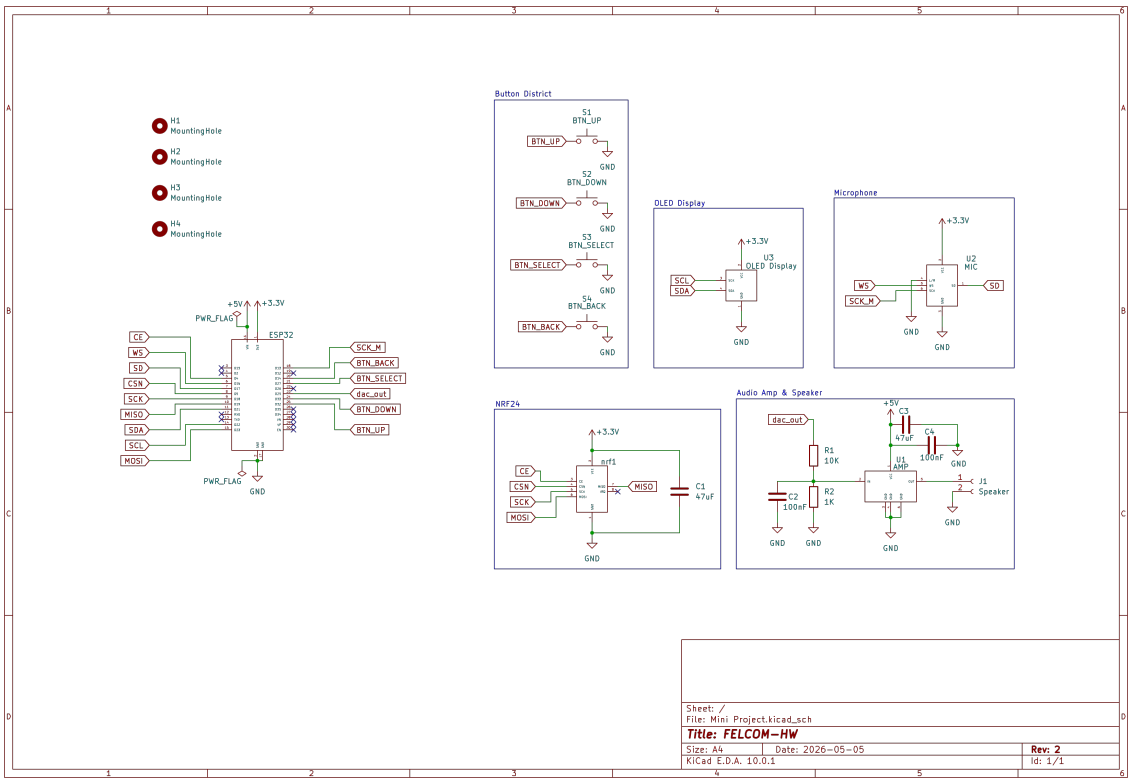

Figure 3: Full hardware schematic of a FELCOM unit (ESP32, nRF24L01+, INMP441, audio amplifier, OLED and controls).

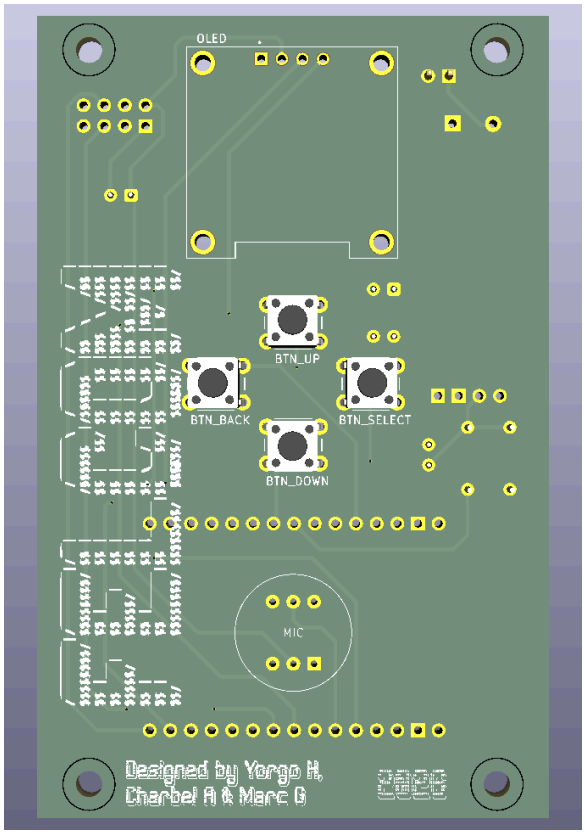

Figure 4: The custom-designed FELCOM PCB layout.

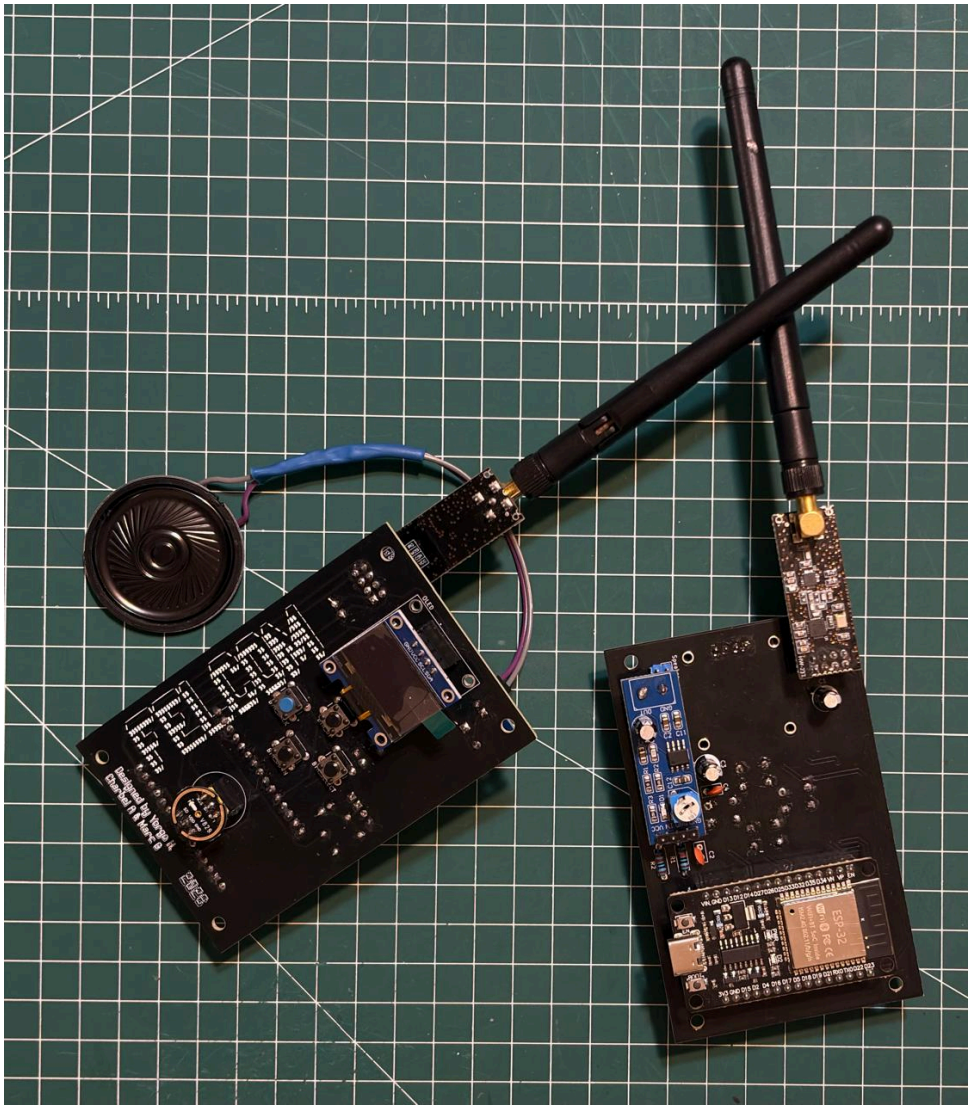

Figure 5: Image of 2 FELCOM units fully assembled

2.2 Firmware Architecture

The firmware is organised in clear layers, which keeps each block (FHSS, framing, FEC, audio) independent and testable:

- **Physical layer**: the RF24 driver and the raw nRF24L01+ register access.
- **Transceiver layer** (`lib/transceiver`): owns the radio. It performs channel access (CSMA), address filtering, and all error-control logic. Crucially, the FEC subsystem is built into the transceiver: application code never calls FEC routines directly, it simply uses `write`, `writeReliable`, `read`, `audioTx` and `audioRx`.
- **Application layer**: chat, audio walkie-talkie, Pong, and the RF/BER test, each with its own payload struct and OLED screen.

A deliberate design decision shapes everything above the radio: the main loop is cooperative and non-blocking. Channel hopping, UI input, message handling and audio capture/playback are all short steps pumped from a single `loop()`. A hardware timer increments a free-running counter every 500 ms, the loop reads it and hops when it

changes. Because no step blocks, hopping always happens on time even while audio is streaming, which is the central constraint the whole architecture is built around.

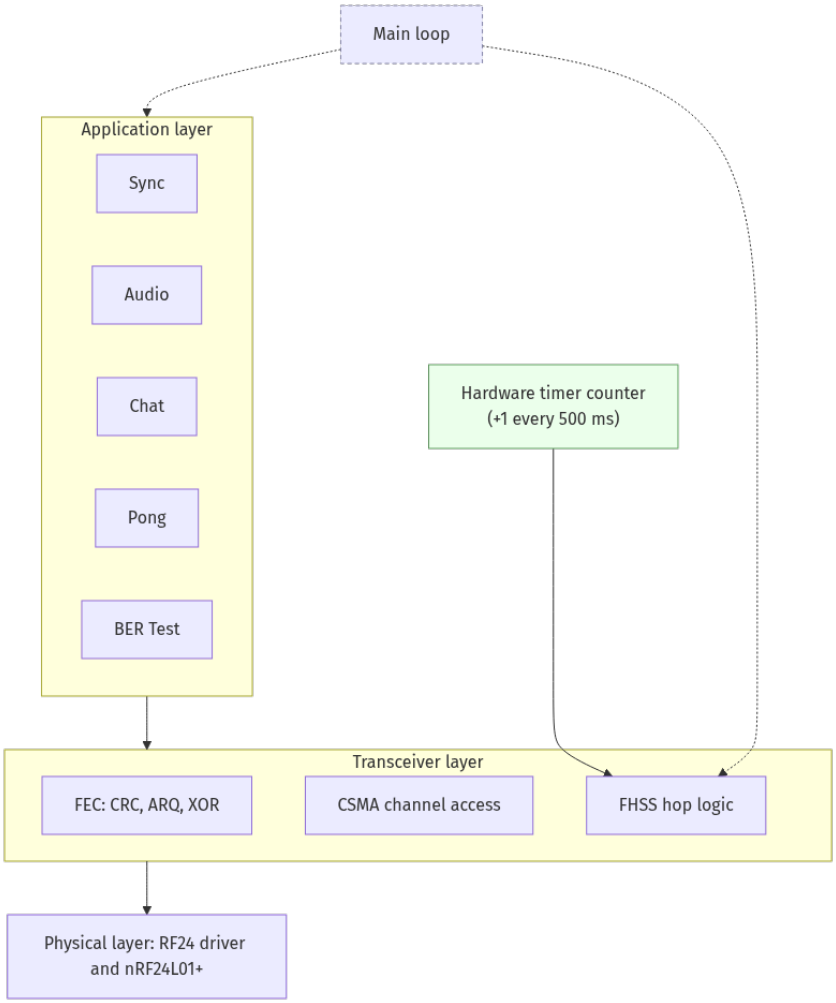

Figure 6: Layered firmware architecture and the non-blocking main loop.

3 Frequency-Hopping Spread Spectrum

FHSS is the heart of the project and the source of its jam and detection-resistance. The nRF24L01+ does the modulation, the firmware keeps changing which of its 125 channels is active.

3.1 Channel Selection

The 2.4 GHz band is crowded, mostly by Wi-Fi, whose channels are 20 to 22 MHz wide. Hopping blindly would land many hops inside a Wi-Fi channel and lose them. We therefore restrict hopping to the top of the band (around 2.51 GHz, nRF channels in the 110 to 115 range), which sits above the standard Wi-Fi allocations and is normally quiet. The current build uses a deliberately small hop set of six channels, `{110, 111, 112, 113, 114, 115}`, which trades a little spreading gain for far easier and faster synchronization.

3.2 Timer-Driven Hopping

Nodes share the same hop schedule. A hardware timer increments a global counter every 500 ms, the active channel is simply `HOPPING_CHANNELS[counter mod 6]`. Because both units derive the channel from the same counter value, they stay on the same frequency as long as their counters agree. A full sweep of the six channels takes 3 seconds.

```
// Every 500 ms the shared timer counter ticks; the channel follows it.
uint32_t slot = readCounter() % HOPPING_CHANNELS_SIZE;
xcvr.setChannel(HOPPING_CHANNELS[slot]);   // one of 110..115
```

Listing 1: Timer-driven hopping: both nodes pick the channel from the same free-running counter, so they stay aligned without exchanging anything.

3.3 The Synchronization Problem

Synchronization is the hardest part of any communication system, and we evaluated the classic approaches and their failure modes:

- **Hop on packet reception**: a receiver that misses a packet stalls on the wrong channel until the transmitter happens to revisit it.
- **Hop on a timer**: robust moment to moment, but the two clocks slowly drift apart, and once they diverge they may never re-align on their own.

Both also share the **new-node** problem: a unit powering on has no idea where in the schedule the network currently is.

In practice the two units are aligned from the dedicated sync screen before a chat or audio session. Clock drift has been deemed as acceptable, from testing no clock drift happened after 3 hours of use.

3.4 Channel Access (CSMA)

Because several packet types share the link, ordinary transmissions use Carrier Sense Multiple Access: before sending, the radio briefly listens, if the channel is busy it backs off a random 1 to 10 ms and retries (up to five times). Real-time audio is the one exception, it deliberately bypasses CSMA backoff, because millisecond stalls would starve the audio pipeline.

4 The Packet Protocol and Data Link

Every transmission, regardless of type, is a fixed **32-byte** frame, the maximum nRF24L01+ payload. A single frame format multiplexing all traffic is what lets text, voice and control share one hopping radio.

4.1 Packet Format

src_node_id	dst_node_id	packet_type	fec	data
1 Byte	1 Byte	4 bits	12 bits	28 Bytes

Figure 7: Packet Structure

The `packet_type` and `fec` fields are packed together into a single 16-bit bit-field, keeping the header to exactly 4 bytes and leaving 28 bytes of payload. Addressing is by one-byte node ID, with `0xFF` reserved for broadcast. This is how, for example, an RF test ping or a sync beacon reaches every node at once.

```
struct __attribute__((packed)) FHSSPacket {
    uint8_t  src_node_id;       // sender
    uint8_t  dst_node_id;       // 0xFF = broadcast
    uint16_t packet_type : 4;   // PONG / CHAT / AUDIO / ND_SYNC / TEST / ACK
    uint16_t fec : 12;          // scheme | XOR block id | slot index
    uint8_t  data[28];          // payload (CRC in the last 2 bytes)
};
```

Listing 2: The single 32-byte frame; `packet_type` and `fec` are packed as bit-fields so the header is just four bytes.

4.2 The Payload Region and the FEC Convention

A strict, project-wide convention governs the 28-byte `data` region so that the framing and the error-control layer never fight over the same bytes:

- The frame size **never** changes; it is always 32 bytes on air.
- For **protected** packets, the last 2 bytes of `data` hold a CRC-16, leaving 26 user bytes. Reliable (ARQ) packets further spend the first 2 of those on a sequence number, leaving 24 application bytes.
- The 12-bit `fec` header field carries **metadata only** (which scheme, and the XOR block/slot identifiers), never the CRC itself, which would not fit in 12 bits.

This single, fixed layout is what allows one `read()` path to demultiplex six different packet types and hand each application exactly its own payload.

5 Forward Error Correction

The radio's own CRC and auto-acknowledgement are **disabled**; error control is handled entirely by our own FEC subsystem, which lives inside the transceiver. This was a conscious choice: different traffic types need very different guarantees, and doing it ourselves lets us pick the right mechanism per type. The subsystem combines three complementary mechanisms.

5.1 The Three Mechanisms

- **CRC-16-CCITT (detection).** Every protected packet carries a 16-bit checksum in the last two payload bytes. On receipt the CRC is recomputed; a mismatch means

corruption and the packet is silently dropped. This turns a noisy link into a "clean or nothing" one.

- **ARQ (retransmission).** For chat, which is infrequent but must be exact, the sender attaches a sequence number, transmits, and **waits for an ACK**, retransmitting on a 200 ms timeout up to three times. The receiver only ACKs a message after it has safely buffered it, so a lost frame is re-sent rather than falsely confirmed. ARQ guarantees delivery but blocks briefly, so it is used only where latency does not matter.
- **XOR block code (forward recovery).** For audio, which is real-time and where waiting for a retransmission is pointless, the sender groups every four data packets and transmits a fifth **parity** packet equal to their bitwise XOR. If any **one** of the five packets in a block is lost, the receiver reconstructs it by XOR-ing the four it did receive. Losses are repaired in the forward direction only.

```
// TX: accumulate a running parity across the data slots of a block.
for (uint8_t b = 0; b < FEC_USER_SIZE; b++)
    parity[b] ^= pkt.data[b];

// After FEC_XOR_BLOCK_SIZE (=4) data packets, emit the parity packet.
if (++count >= FEC_XOR_BLOCK_SIZE)
    transmitAudioPacket(parityPkt);   // lets RX rebuild one lost packet
```

Listing 3: The XOR parity is just the bitwise XOR of the four data packets; the receiver rebuilds any single missing packet from the other four plus the parity.

5.2 Per-Type Policy

Each packet type is matched to the mechanism that fits its needs:

Type	CRC	ARQ	XOR	Rationale
CHAT	✓	✓	✗	Infrequent; must be exact.
AUDIO	✓	✗	✓	Real-time; forward recovery only.
PONG	✓	✗	✗	Frequent; ARQ would backlog.
ACK	✓	✗	✗	Small control packet.
ND_SYNC	✗ (raw)	✗	✗	Must stay forgiving for resync.
TEST	✗ (raw)	✗	✗	Must measure **real** bit errors.

The TEST packet is sent completely raw on purpose, so the BER screen measures the true error rate of the channel rather than a CRC-cleaned version of it.

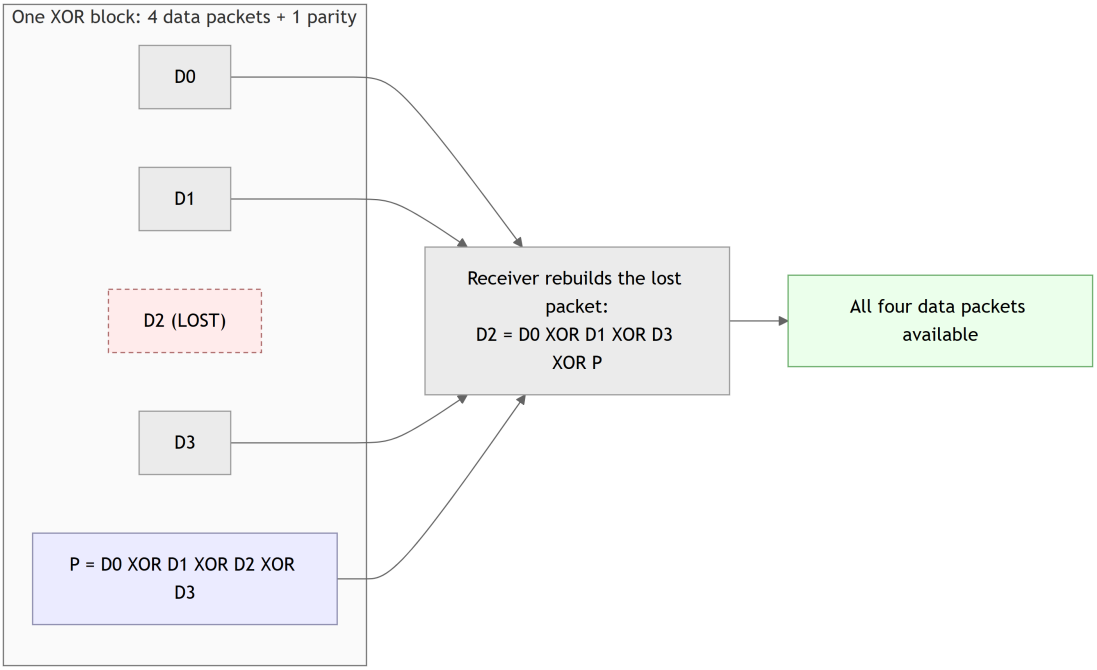

Figure 8: XOR block forward error correction: one lost packet per block of four is reconstructed from the parity packet.

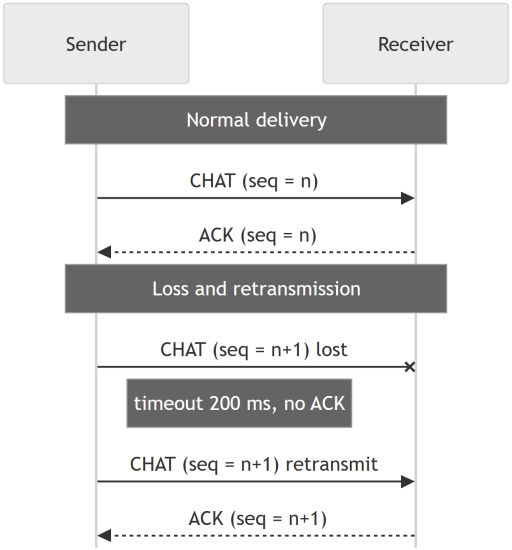

Figure 9: ARQ exchange for reliable chat delivery, including retransmission on timeout.

6 Real-Time Audio

The audio mode turns the pair into a half-duplex **walkie-talkie**: one node TALKS, the other LISTENS, and Select toggles the role. It is the most demanding feature, because live voice must coexist with channel hopping that interrupts the link every 500 ms.

6.1 Format and Capture

Voice is captured from the INMP441 as **8 kHz, 8-bit mono PCM**, deliberately low-fidelity, which is enough for intelligible speech and keeps the data rate low. The microphone is read over I^2S without blocking. Each sample is then conditioned in

software: a running DC-offset estimate is subtracted (an exponential moving average), a gain factor is applied, and the result is clamped to 8 bits. Twenty-four conditioned samples (3 ms of audio) are packed into one `AudioPayload`, which is sized to **exactly** fill the 26-byte protected user region.

6.2 Transport and Playback

Each payload is handed to `audioTx`, which wraps it with CRC + XOR-block FEC and transmits it (skipping CSMA backoff, as noted earlier). On the listening side, recovered payloads come back through `audioRx` and are written into a large ring buffer that absorbs network jitter; samples are then drained to the DAC (GPIO25) at a precise 8 kHz using `micros()` timing. No extra hardware timer is needed, since the hop counter already owns one.

6.3 Non-Blocking Pipeline

The whole chain (capture, condition, transmit, receive, play) is a set of short cooperative steps pumped from the main loop. Nothing blocks, so FHSS hopping continues uninterrupted while audio flows. The XOR FEC matters most here: at a hop boundary the in-flight packet is often lost, and the block code transparently rebuilds it, smoothing what would otherwise be an audible click every half second.

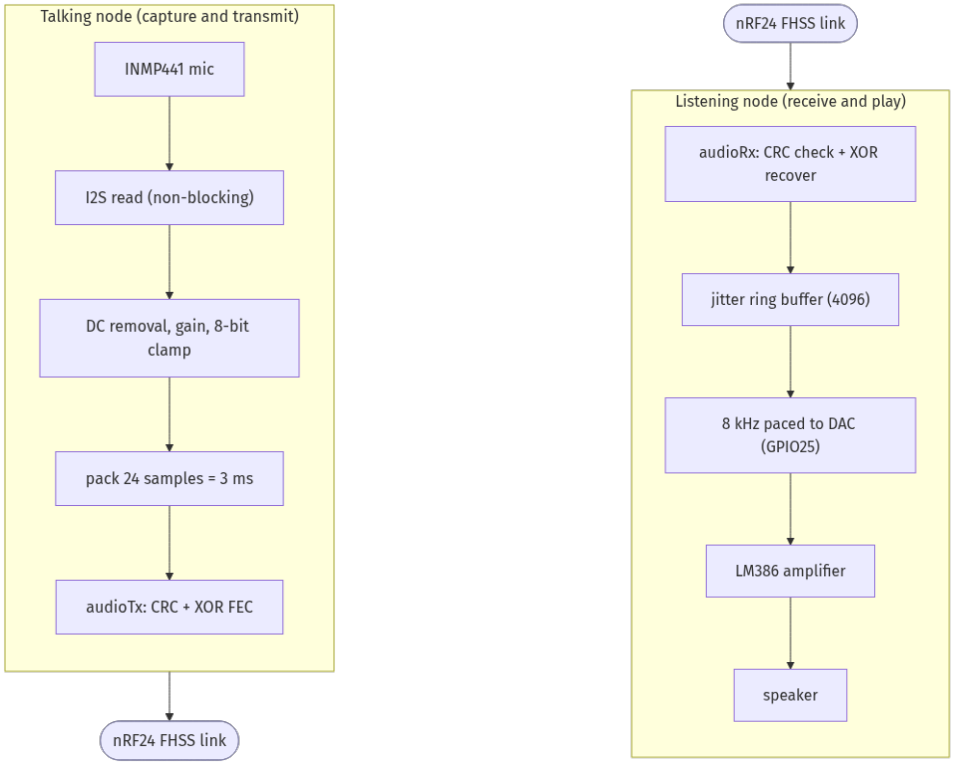

Figure 10: The non-blocking real-time audio pipeline, from microphone to speaker.

7 Security and Applications

7.1 Text Confidentiality

Chat text is passed through a lightweight **XOR stream cipher** (a repeating 4-byte key) before transmission and reversed on receipt, so the message is not sent in clear over the air. Combined with the unknown hopping sequence, which already makes the traffic hard to capture coherently, this gives a basic layer of confidentiality appropriate to the platform. It is worth noting that this is obfuscation, not strong cryptography; a stronger, authenticated cipher would be a clear improvement in terms of security.

7.2 Applications

Four screens exercise the stack:

- **Chat**: encrypted text, sent point-to-point (reliable: ARQ + CRC) or broadcast (CRC only), shown on a scrolling OLED log.
- **Audio**: the half-duplex walkie-talkie of Chapter 6.
- **Pong**: a two-player real-time game whose paddle/ball updates validate low-latency **bidirectional** traffic over the hopping link.
- **RF / BER Test**: a diagnostic mode (next chapter).

8 Testing, Results and Analysis

8.1 Method

Two instruments are built into the firmware. The **RF/BER test** sends raw, known-pattern packets (`0xAB` repeated) at a fixed rate; the receiver echoes them back, and both sides count sent / received / echoed / corrupt packets. Because TEST packets bypass the FEC, this measures the **true** channel error rate. Second, the transceiver keeps live **FEC counters** (CRC pass/fail, ARQ sent/retransmitted/acked/failed, and audio blocks/recovered/lost), printed as a throttled serial summary. Together they let us separate raw link quality from what the FEC layer recovers.

8.2 Results

TODO (insert before submission): Insert the measured numbers from your test runs. Suggested figures/tables: (a) packet-loss and corruption rate from the RF/BER screen, fixed channel vs. hopping; (b) FEC counters during a chat session (CRC ok/bad, ARQ retransmissions); (c) audio session counters (blocks finalised, packets recovered by XOR, packets lost); (d) any jamming experiment: loss with a narrowband interferer present, with and without hopping. Replace the placeholder below with the real chart/table.

Diagram placeholder (to be drawn)

Results chart/table: e.g. bar chart of packet-loss % (fixed-frequency vs. FHSS, with and without an interferer), or a table of the FEC counters captured during a representative chat and audio session.

Figure 13: Measured link reliability and FEC recovery results.

8.3 Analysis

TODO (insert before submission): Write 1 to 2 paragraphs interpreting the numbers once measured: how much loss FHSS avoids under interference, how often XOR FEC rebuilds audio packets (especially at hop boundaries), how many ARQ retransmissions chat needs, and where the link still struggles (e.g. after clock drift).

9 Challenges and How We Tackled Them

- **Synchronization**: The most bothersome challenge we faced. After many failed implementations we settle on a timer based synchronization where nodes deliberately send a sync packet and synchronize together.
- **pong** buggy
- **audio** inaudible.
- **CSMA stalling real-time audio.** The 1 to 10 ms CSMA backoff, fine for chat, starved the audio pipeline and caused choppy playback. Audio transmission was made to bypass backoff; collisions are tolerated because the mode is half-duplex and already FEC-protected.

10 Open issues

- Synchornization is manual
- nRF24L01+ is not a realiable radio module

11 Conclusion

FELCOM shows that resilient, jam-resistant, infrastructure-free communication, both text and live voice, can be built on inexpensive, commodity hardware. The working system hops across six interference-free channels under software control, keeps two nodes aligned on a shared hopping schedule, multiplexes several traffic types over one 32-byte frame, and matches a forward-error-correction mechanism to each: CRC detection everywhere, ARQ for exact chat delivery, and an XOR block code that repairs real-time audio without retransmission. The non-blocking architecture is what ties it together, letting voice stream while the radio keeps hopping. Natural extensions

include a stronger, authenticated cipher (such as AES) applied to both text and audio, automatic clock-drift correction to remove the manual re-sync step, and a move beyond the point-to-point link toward a small multi-node network.

12 References

[1] Nordic Semiconductor, *nRF24L01+ Single Chip 2.4 GHz Transceiver Product Specification.*

[2] Espressif Systems, *ESP32 Technical Reference Manual* and *ESP32 Series Datasheet.*

[3] InvenSense (TDK), *INMP441 Omnidirectional Microphone with Bottom Port and I^2S Digital Output, Datasheet.*

[4] Texas Instruments, *LM386 Low Voltage Audio Power Amplifier, Datasheet.*

[5] TMRh20, *RF24: Optimized Driver for nRF24L01(+) on Arduino & Raspberry Pi* (open-source library).

[6] H. K. Markey (Lamarr) and G. Antheil, *Secret Communication System*, U.S. Patent 2,292,387, 1942.

[7] ITU-T Recommendation V.41, *Code-independent error-control system* (CRC-16-CCITT).

[8] A. Goldsmith, *Wireless Communications*, Cambridge University Press (spread spectrum and FHSS fundamentals).